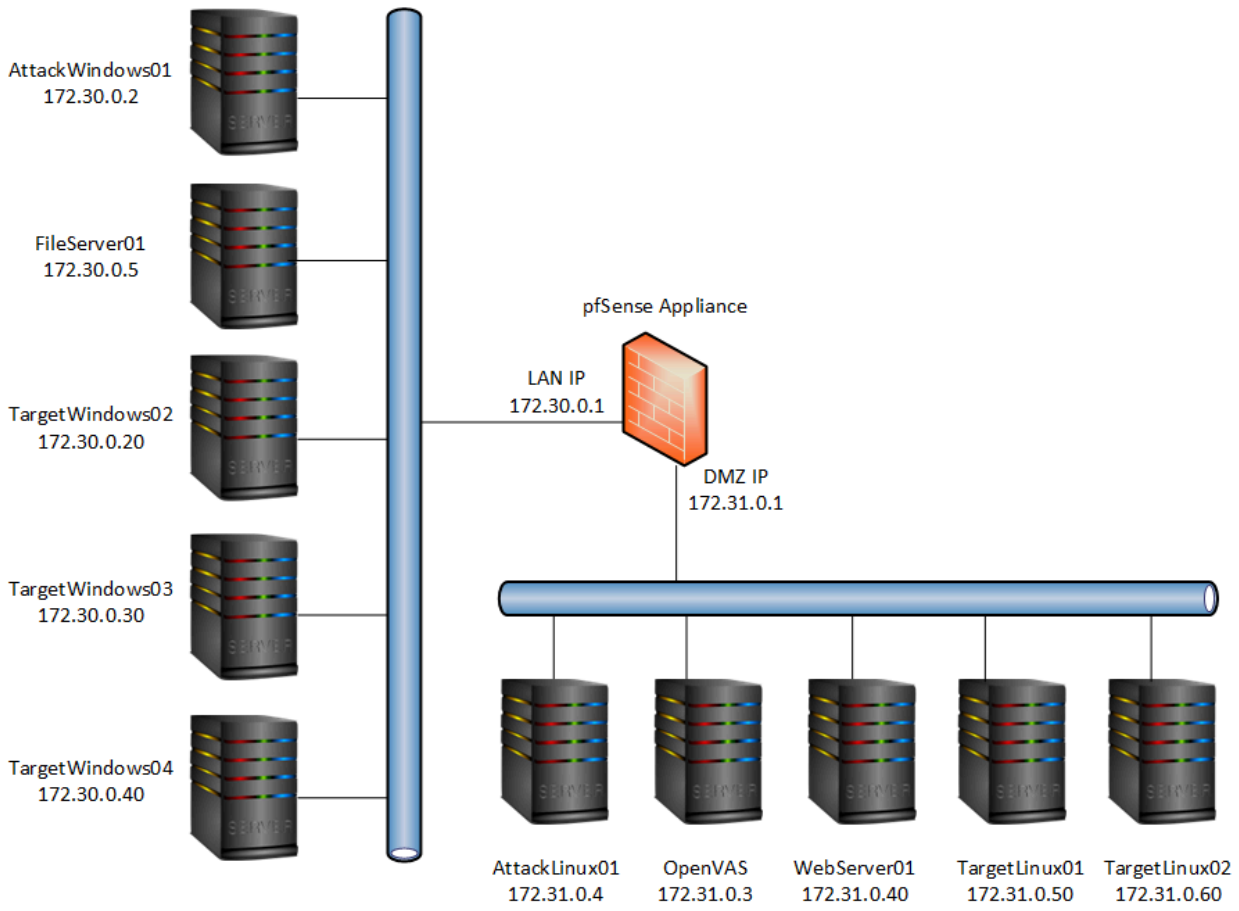
Saminur Karim

Active Reconnaissance

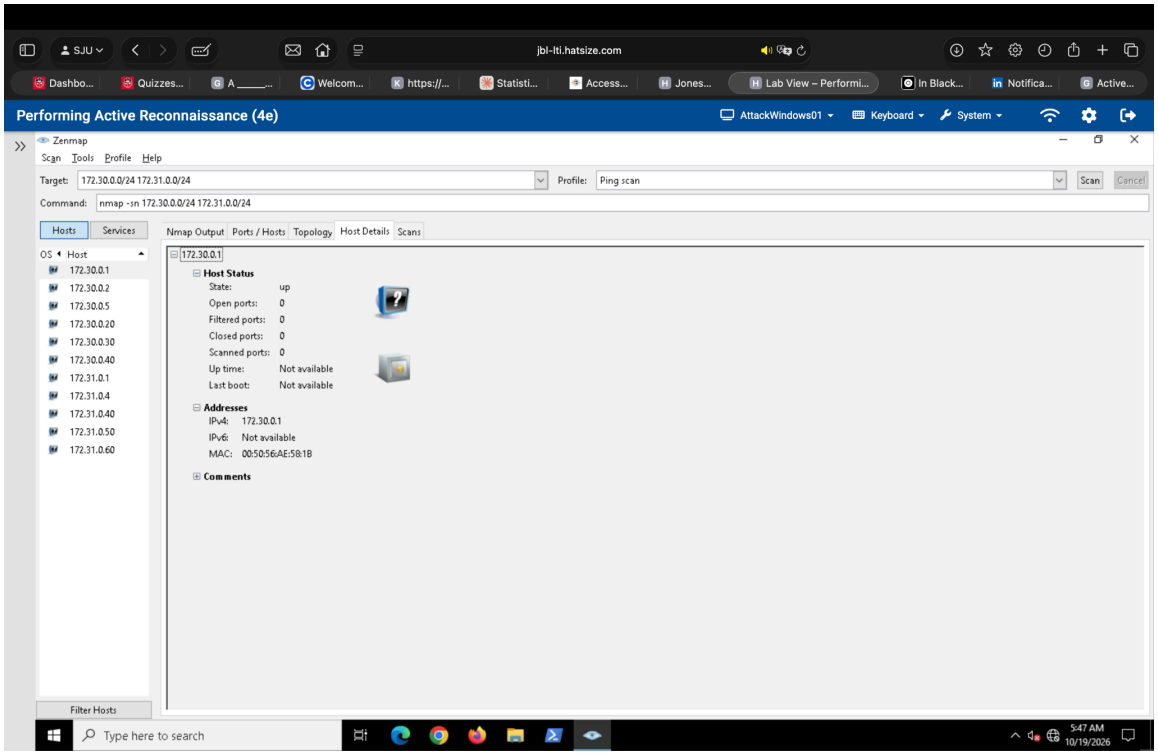
Active reconnaissance is the act of scanning networks to discover new vulnerabilities, identify weakpoints, and test the detection and response of systems. This step often occurs after a passive reconnaissance of the target where information is gathered through OSINT. A port scanner such as Zenmap reveals information such as OS details, open ports and services, host details, and network topology. Most networks use firewalls and detection systems that receive logs of activity. This form of reconnaissance leaves details of the scan and may be detected. Network perimeter devices can be configured to distinguish legitimate access or suspicious activity.

The result of the active reconnaissance is a list of the addresses of machines that are active on the target network. The details for each machine include the ports that are listening for connections, as well as any information that could be determined about the software that is listening on those ports. All of this information is important for attempting to gain access to the network.

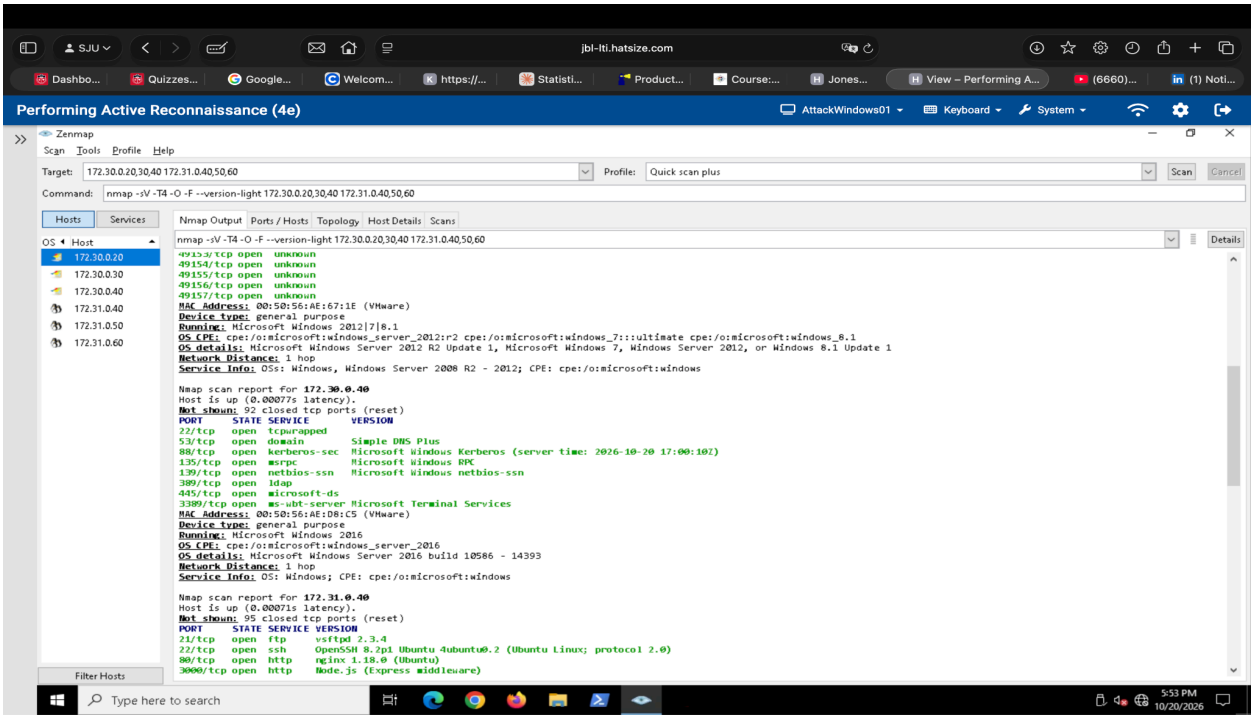
The next stage of the process involves identifying actual vulnerabilities that exist in the target network. This stage is called vulnerability enumeration. This lab contains a Zenmap for network scanning and Nessus for vulnerability scanning. Below is the network topology that displays IP addressing for all endpoints.



This screenshot below displays a Zenmap scan on the 172.30.0.0/24 and 172.31.0.0/24 subnets using a -sn scan. This -sn scan skips port scanning, discovers live hosts, and maps subnets. The program attempts an ICMP ping to each address. Zenmaps does a reverse DNS lookup to determine hostnames. Under host details, I was able to view every endpoint in both subnets.

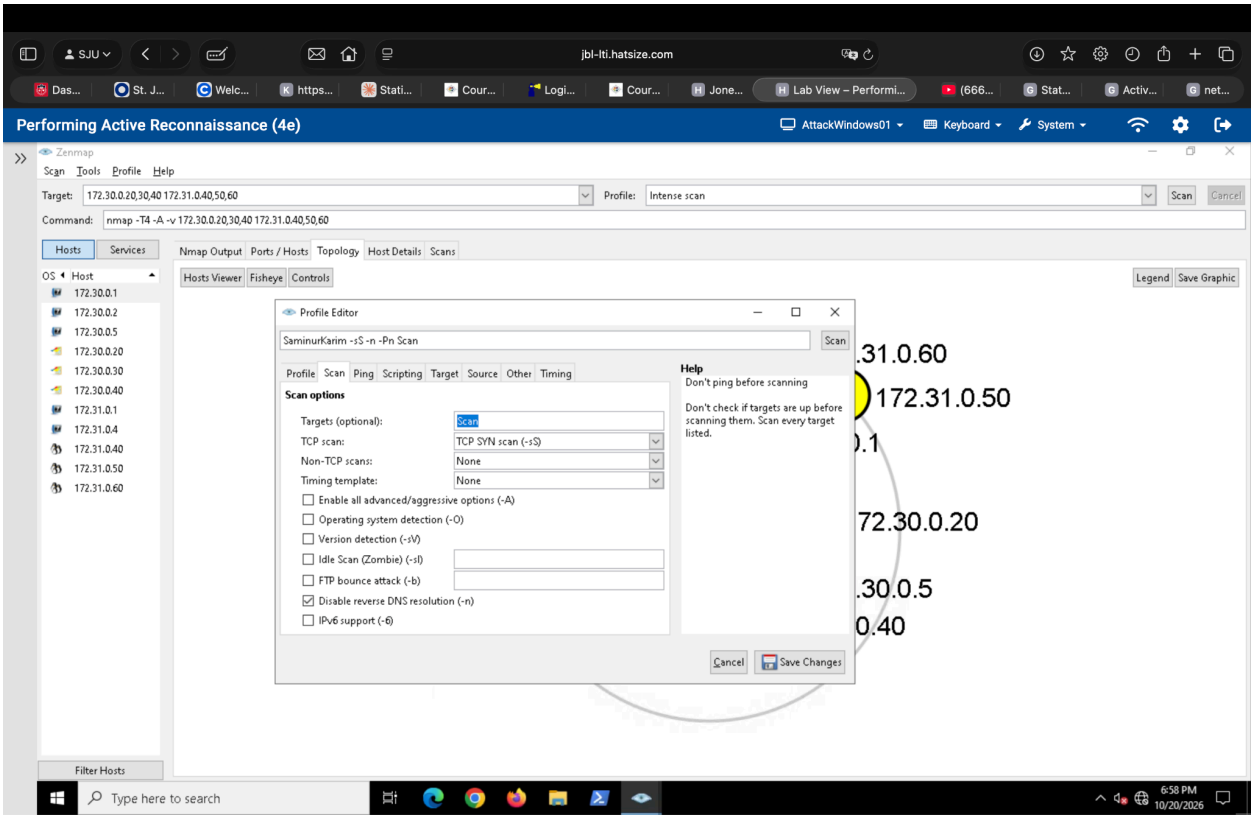


The next scan was a quick scan plus on 172.30.0.20,30,40 172.31.0.40,50,60. This nmap scan contains multiple flags to specify what the scan does. The -sV requests a version detection of the software used in the open ports. The -T4 relates to the scan timing. The -O detects the operating system used by the device. The -F restricts the port scanning to the 100 most common ports.

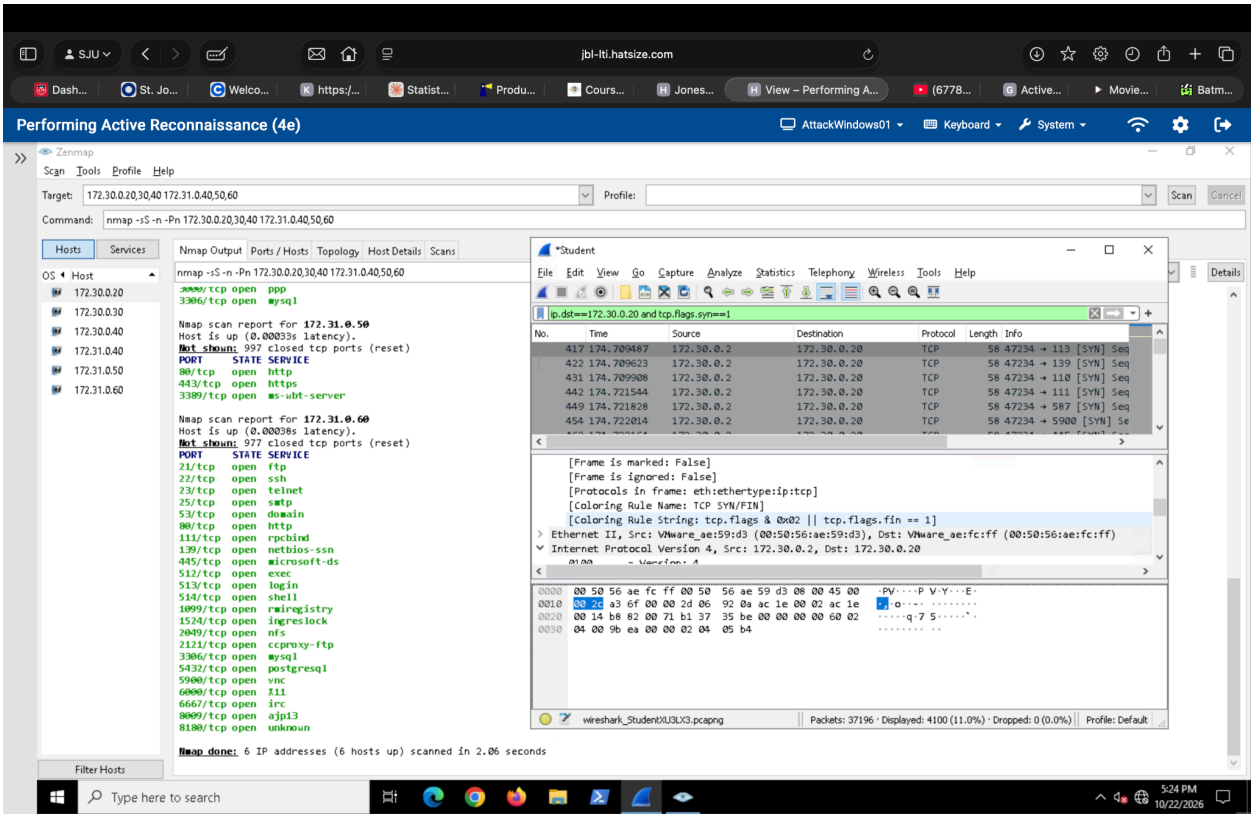


An intense scan contains certain options for more details but with a cost of longer time to finish one. These scans cover more ports which is an option when simple scans are not effective. The -A enables OS detection like -O, as well as version scanning (-sV), script scanning (-sC), and traceroute (--traceroute). The -v requests more verbose output from nmap. It will cause some information to be shown in the Nmap Output tab before the scan is complete. The logical topology can show information such as open ports and what type of device each host is.

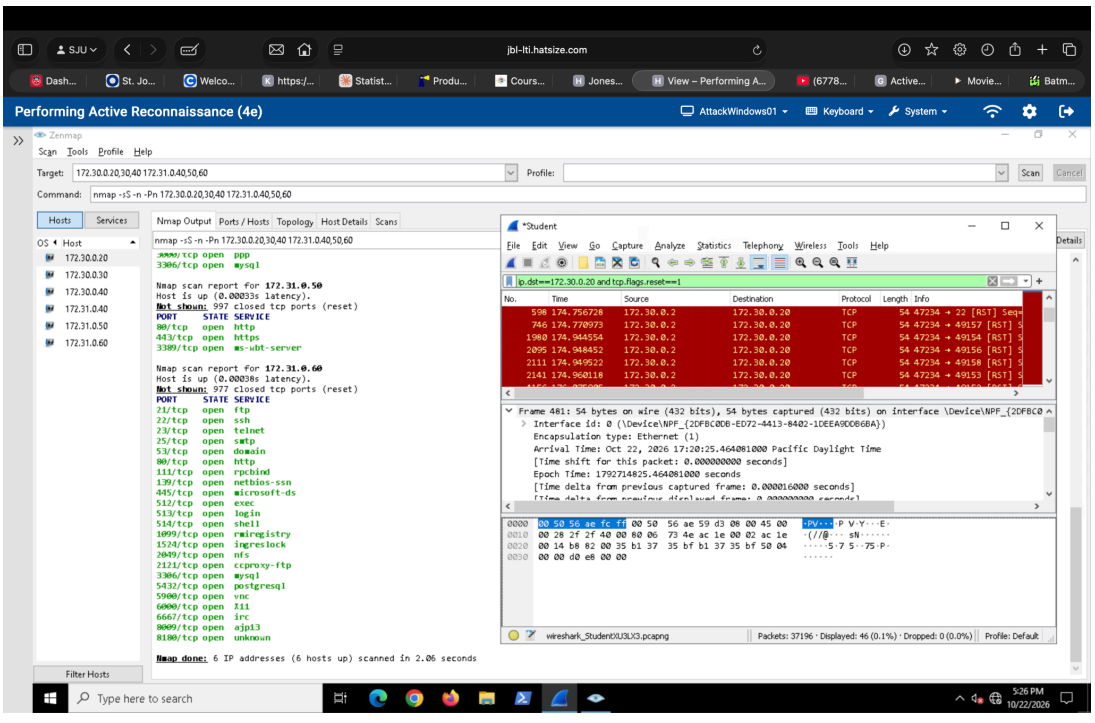
In the next step, I created a scan profile that sped up scans. Flags in packet headers represent the purpose of each packet. I clicked **Profile > New Profile or Command.** Then, in the profile editor I entered the name of the scan. I clicked the scan tab and selected TCP SYN scan (-sS) which avoids a complete connection by only initiating one. I **disabled reverse DNS resolution** to prevent DNS lookups which extend the scan. I clicked the Ping tab and checked **Don't Ping before scanning** to prevent host discovery. I clicked save changes and clicked on the scan in profile.



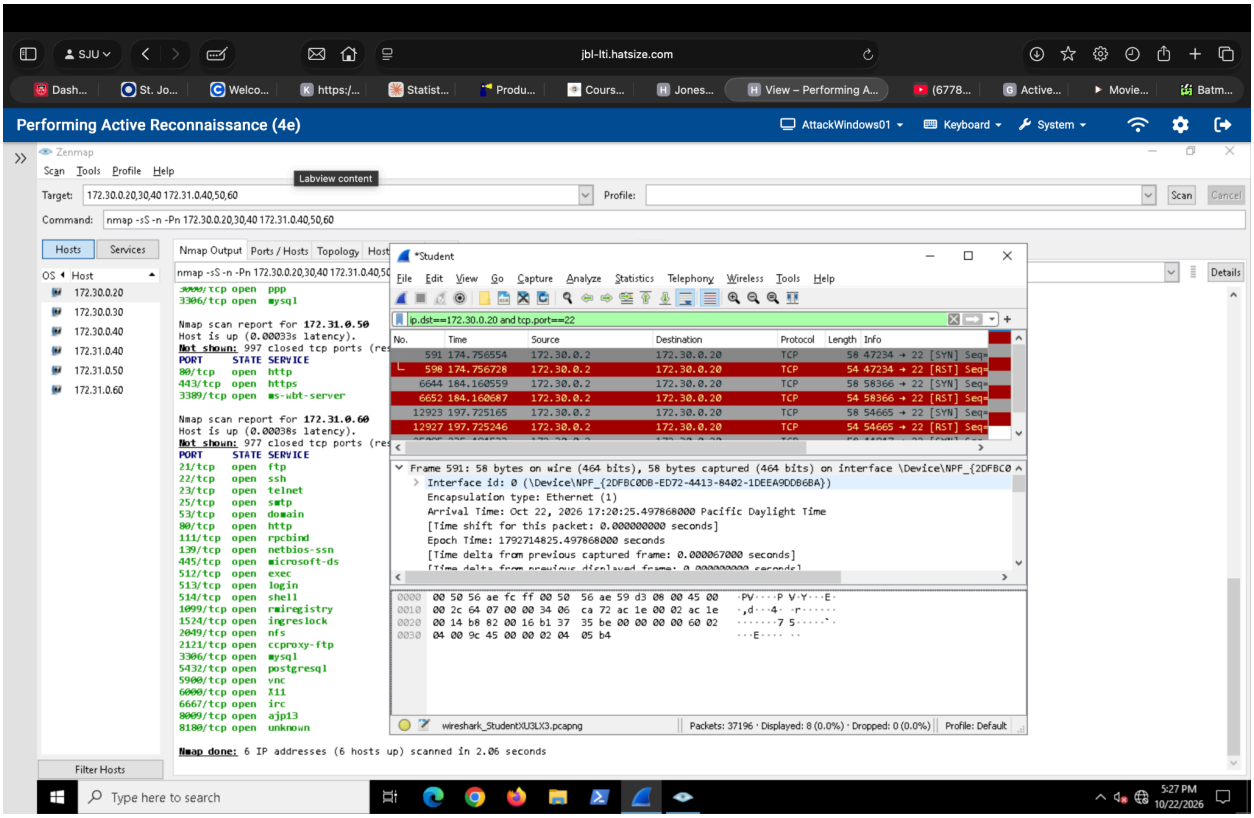
The following step of the lab requires the scan to be performed then using Wireshark to view the contents of the packet like addressing information and key fields in the header. I used Wireshark to view the TCP handshake process and how each port responded to the packet. I ran the scan then added a filter that targets 172.30.0.20 and TCP SYN flags.



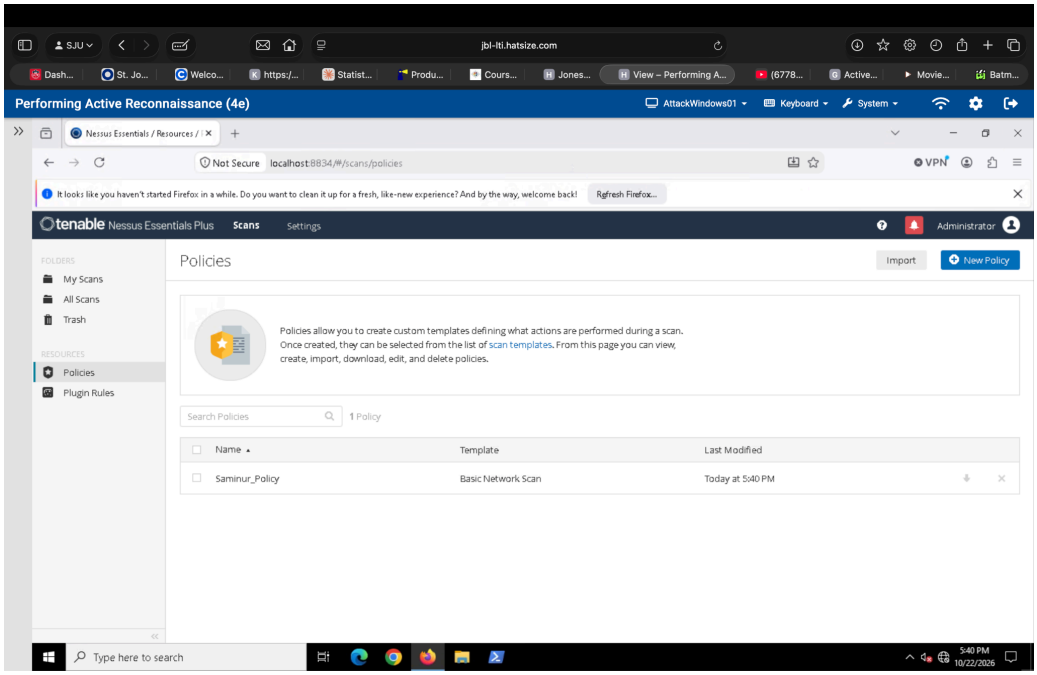
The screenshot below filters for RST flags that terminate the TCP handshake immediately. There were 12 RST packets corresponding to the 12 open ports. Each open port responded with a SYN/ACK packet, and Nmap immediately sent an RST packet to terminate the connection without completing the TCP three-way handshake.



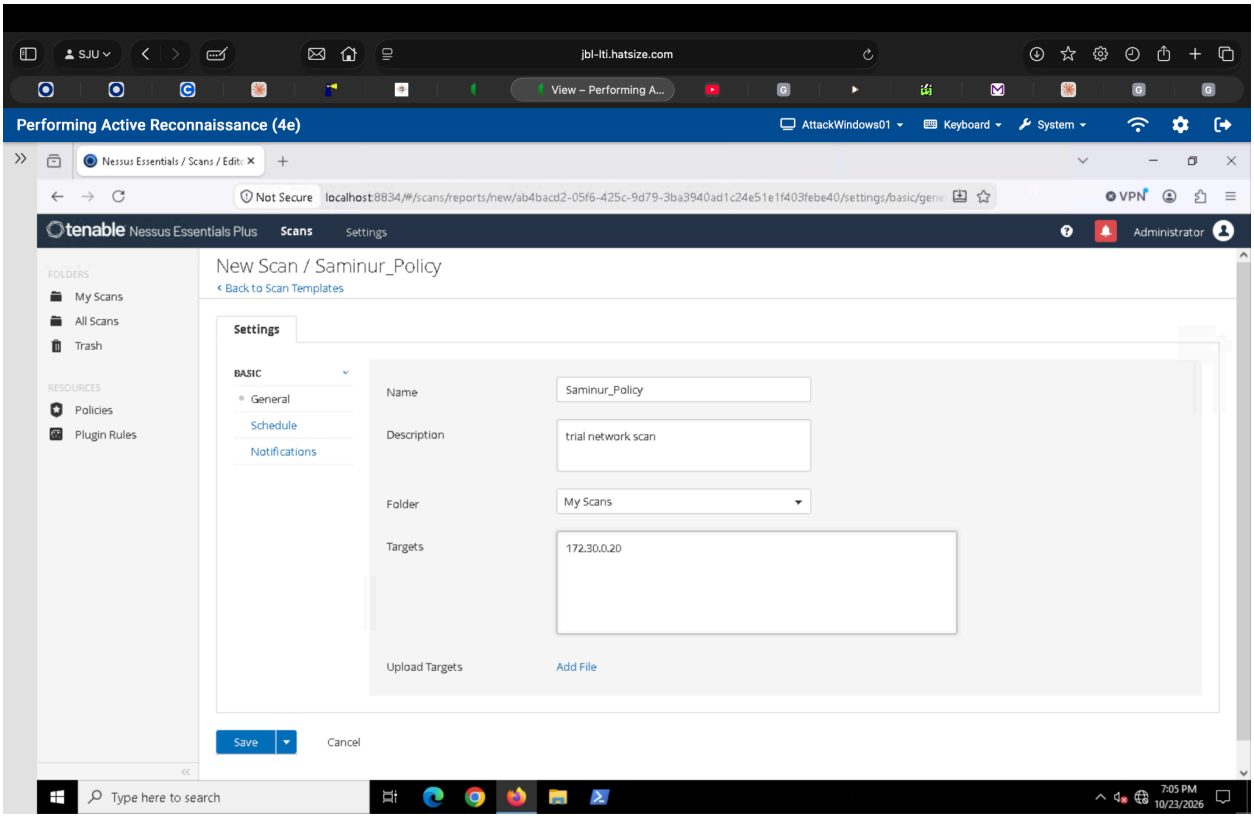
The screenshot below contains all packets in port 22 in the host specified. Port 22 is commonly used for SSH. While SSH itself is a secure remote administration protocol, exposing it to untrusted networks can increase the attack surface if weak passwords, outdated software, or misconfigurations are present.



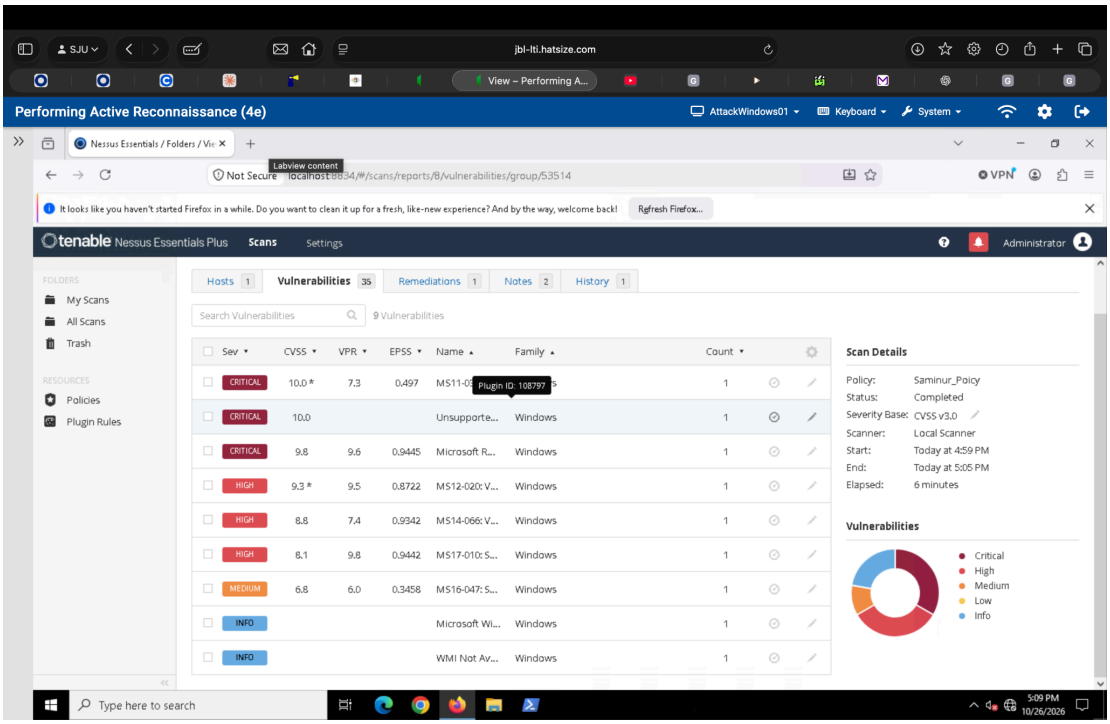
Nessus is a vulnerability scanning platform used by many enterprises. Below, I created a policy to begin a network scan for devices.



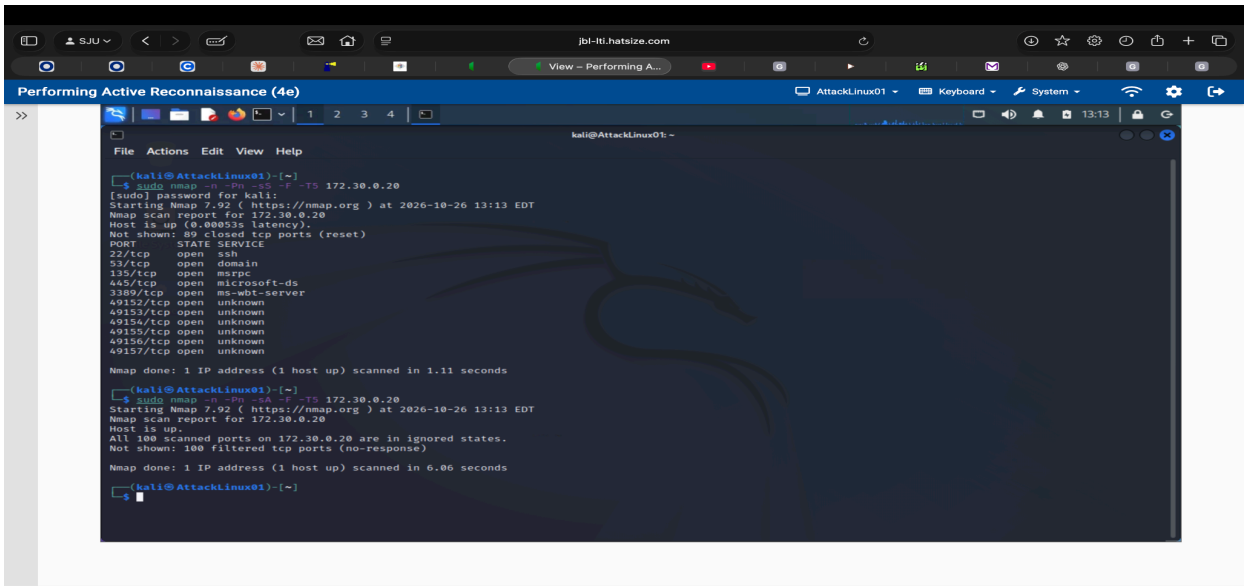
Then, I went to my scans and created a basic network scan and launched it. The target was 172.30.0.20 from earlier for a basic network scan.



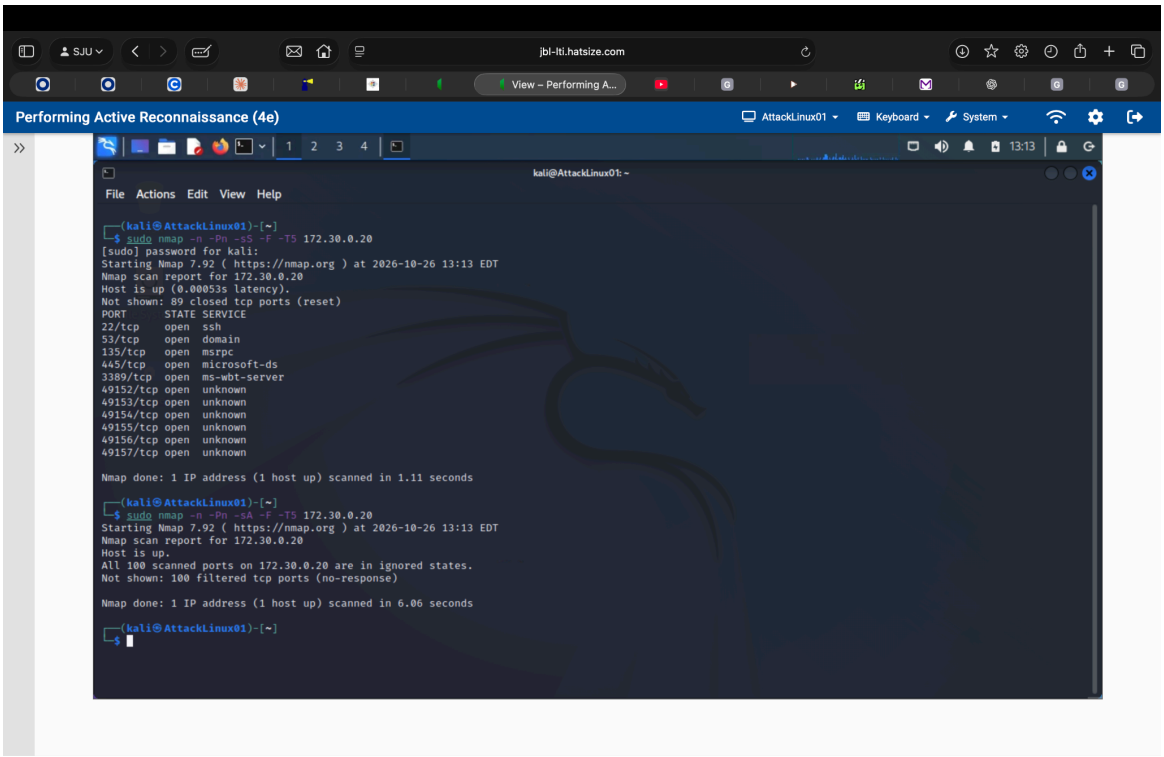
After the scan was complete, vulnerabilities ranging from high to low were discovered. Software, hardware, and application vulnerabilties all had details and remeditations from Nessus.



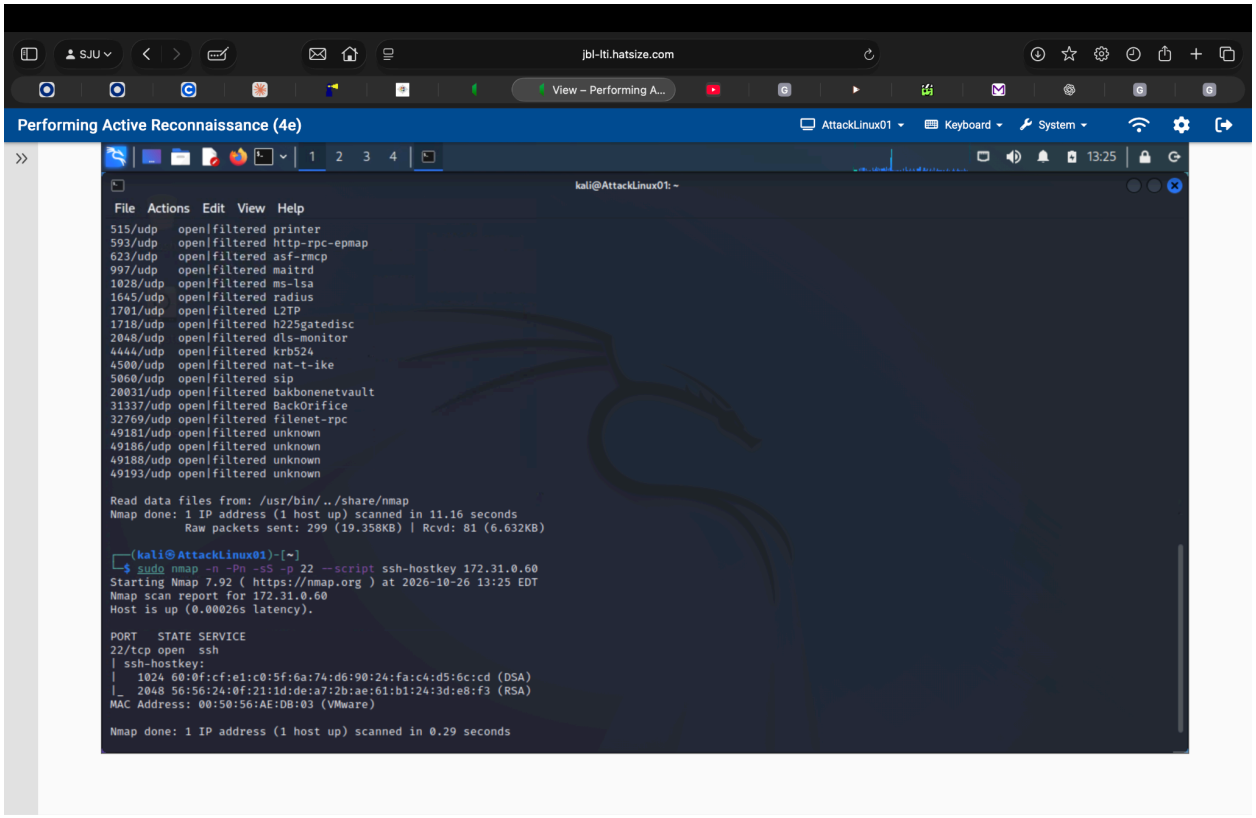
I switched to a Kali attack host that had nmap pre-installed. I ran an aggressive SYN scan and an ACK scan. The command uses -n (disable DNS), -Pn (disable ping scan), -F (limit to common ports), and -T5 (aggressive timing) to speed up the scan. There were 12 open ports and 88 closed ports just like the Zenmap scans. The ACK scan discovered the filtered ports which reveals that a software or hardware device is blocking invalid ACK packets from Kali.



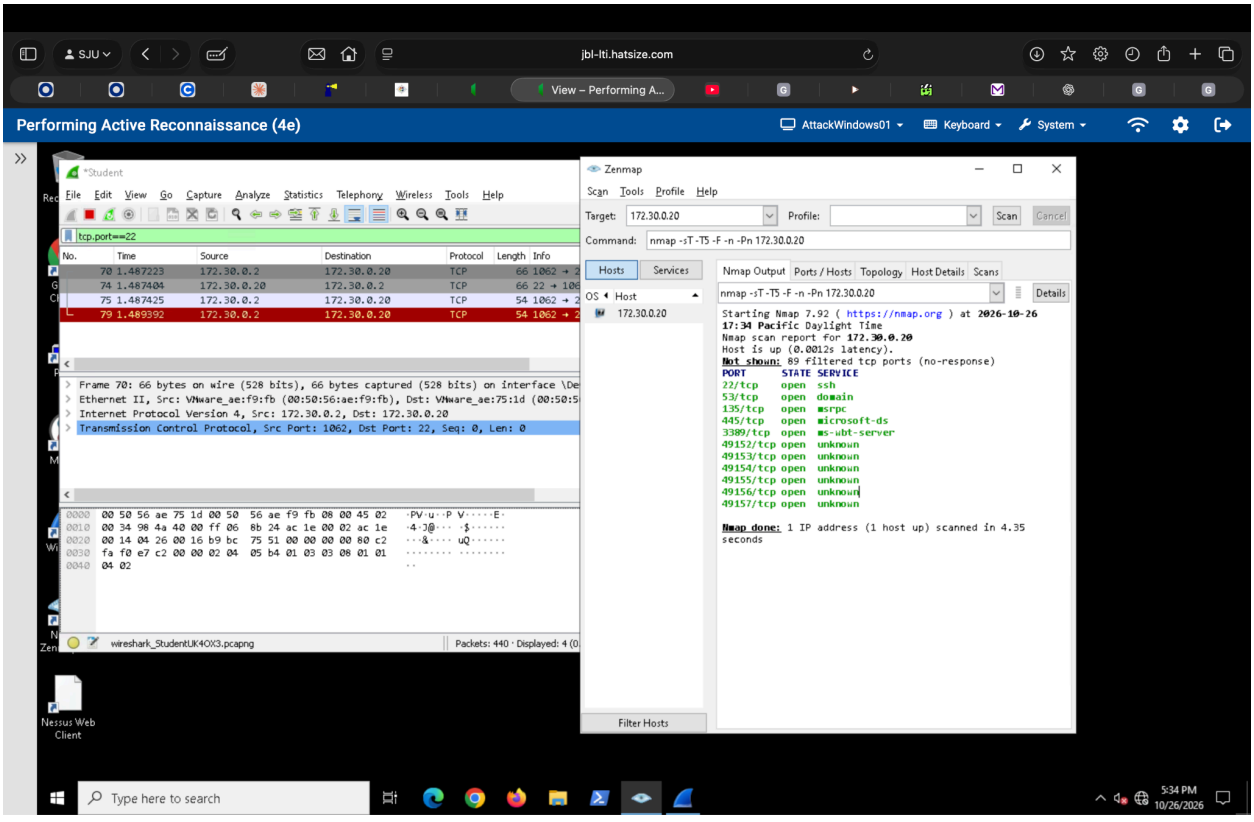
To discover the path to the target, I added traceroute to the scan. This revealed two addresses which can reveal network topology. The traceroute indicated an intermediate network device, which may be a router or firewall. Additional analysis would be required to determine whether it is a stateful firewall.



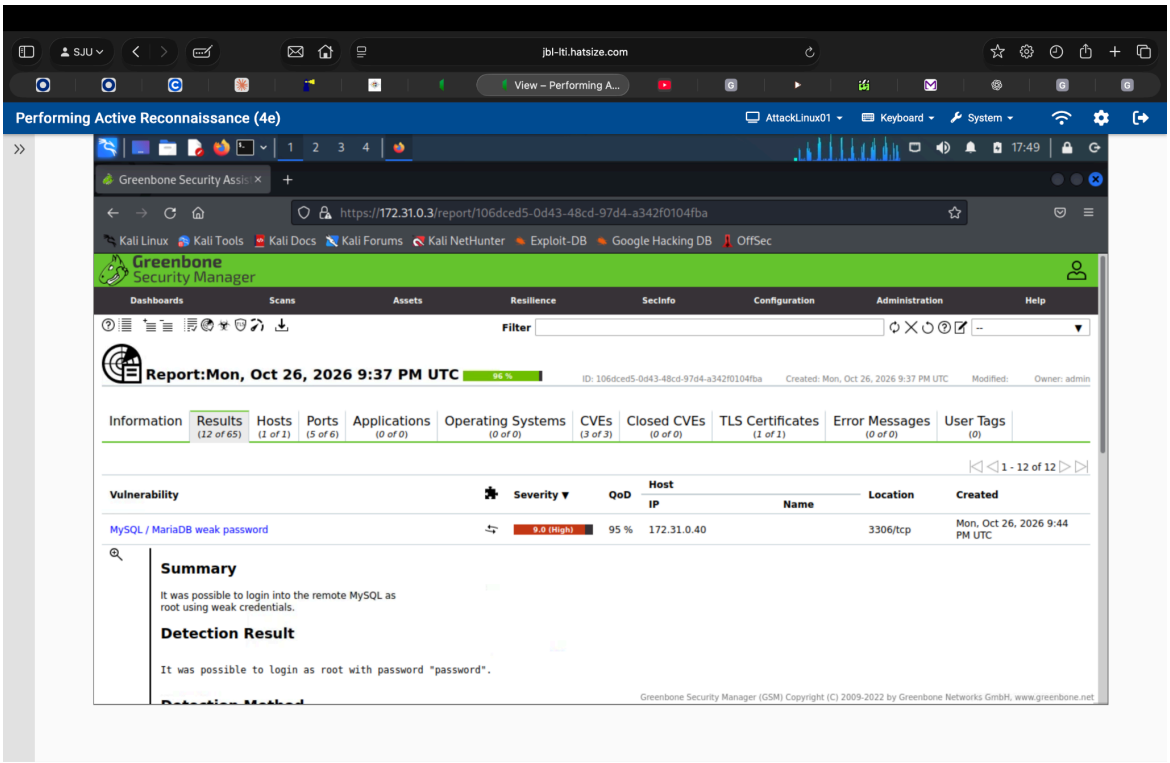
Using the Nmap Scripting Engine (NSE), Nmap can perform service enumeration, configuration auditing, and vulnerability detection. Some NSE scripts test for known vulnerabilities, but they do not generally exploit systems.



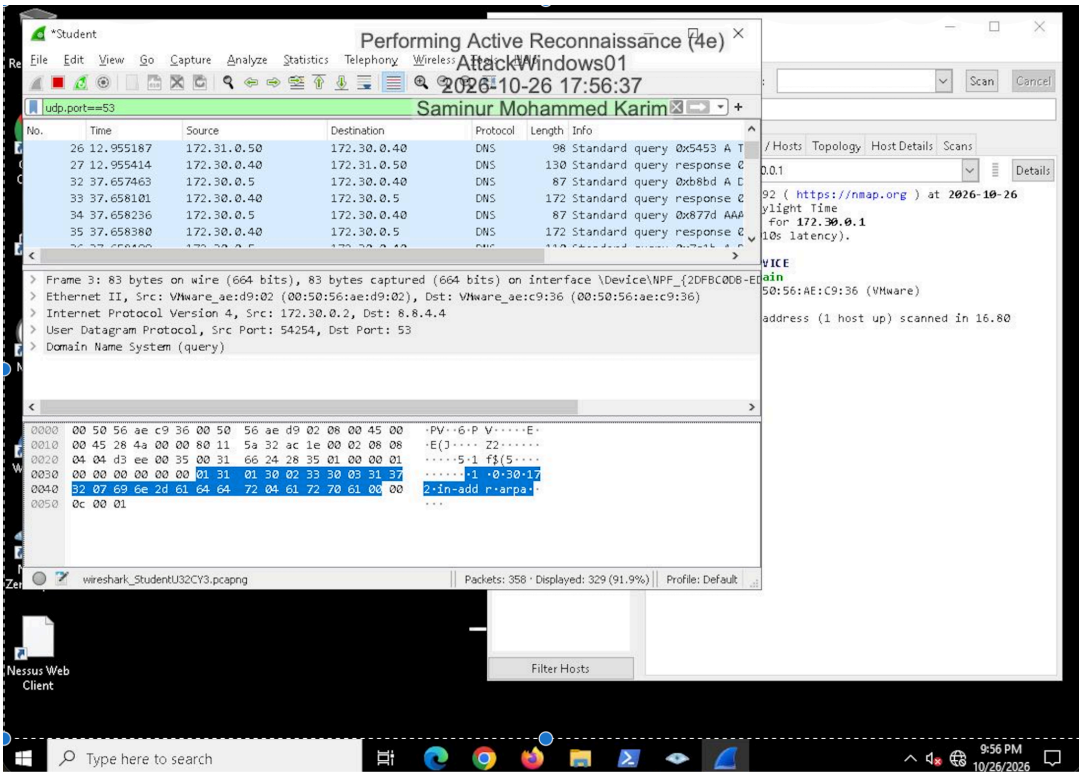
The screenshot below displays a WireShark capture of the TCP connection on port 22. The Wireshark capture shows the TCP three-way handshake (SYN, SYN/ACK, ACK) followed by connection termination.



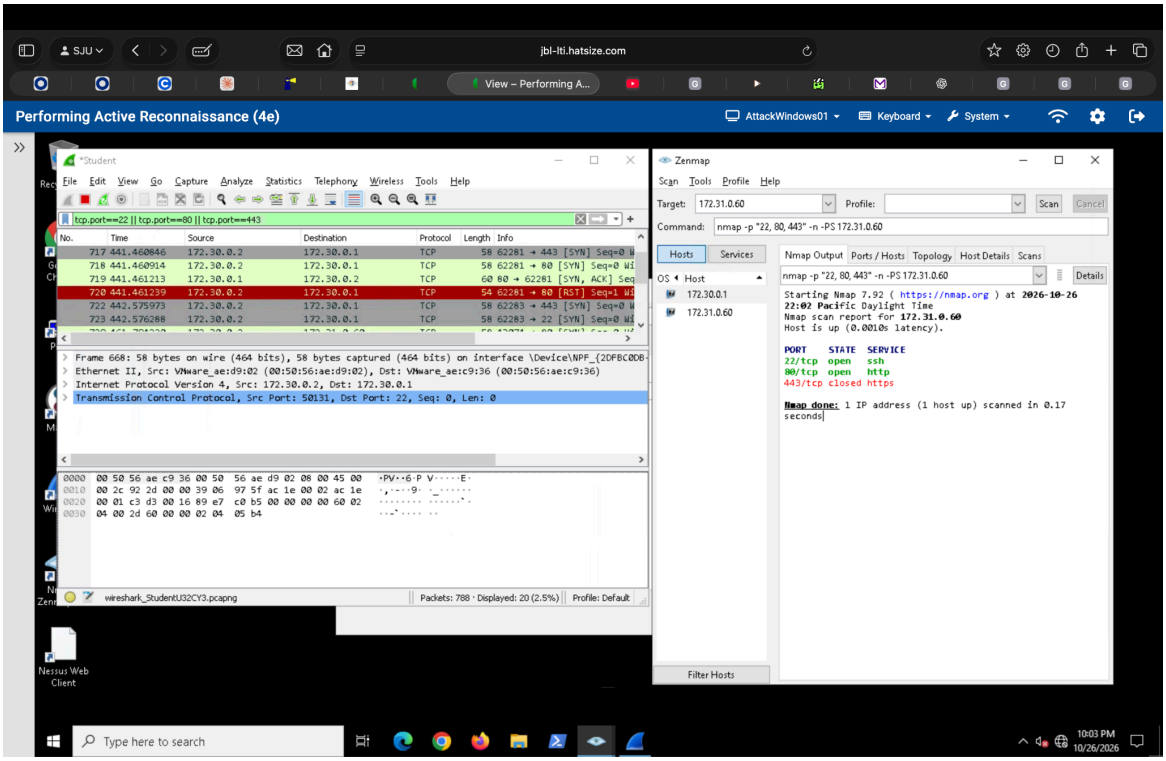
Afterwards, I ran a vulnerability scan on Openvas. There were many vulnerabilities exposed, this one in particular was the highest CVSS score of 9.0. This can be mitigated by having a password length and character requirement with an account lockout policy.



Next, I scanned UDP port 53 using Zenmap. Using WireShark I was able to monitor the open port and inspect the contents of the header and payload. Wireshark displayed the UDP request and response packets, allowing inspection of the source and destination IP addresses, UDP ports, packet length, and payload.



I then created a profile for common scans that would need to be repeated. I removed DNS resolution, targeting TCP port 22, 80, 443, on 172.31.0.60. I identified the contents and the handshake process for all TCP connections from those ports. HTTPS was closed.



CONCLUSION

In conclusion, this lab provided hands-on experience with the tools and techniques used during the active reconnaissance phase of a penetration test. Using Zenmap and Nmap, I identified live hosts, open ports, operating systems, running services, and network topology while comparing different scan types such as SYN, ACK, UDP, and aggressive scans. Wireshark allowed me to analyze the packets generated by these scans and better understand TCP handshakes, UDP communication, and packet headers. I also used Nessus and OpenVAS to perform vulnerability assessments and review the severity, impact, and remediation recommendations for discovered vulnerabilities. Overall, this lab demonstrated how network scanning and vulnerability enumeration work together to identify potential security weaknesses and provided practical experience with tools that are widely used in cybersecurity and penetration testing.